



**DEPARTMENT OF INFORMATION TECHNOLOGY
LAB MANUAL**

CS23432 – Software Construction

(REGULATION 2023)

**RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015**

Name: Spurgen A

Register No: 2116241001257

Year / Branch / Section: II / B.Tech IT

Semester: IV

Academic Year: 2025-2026

INDEX

S.No.	Date	Title
1.	21/01/26	Azure Devops Environment Setup.
2.	28/01/26	Azure Devops Project Setup and User Story Management.
3.	04/02/26	Setting Up Epics, Features, And User Stories for Project Planning.
4.	11/02/26	Sprint Planning.
5.	18/02/26	Poker Estimation.
6.	25/02/26	Designing Class and Sequence Diagrams for Project Architecture.
7.	11/03/26	Designing Architectural and ER Diagrams for Project Structure.
8.	18/03/26	Testing – Test Plans and Test Cases.
9.	25/03/26	Load Testing and Pipelines.
10.	08/04/26	GitHub: Project Structure & Naming Conventions.

EXP NO: 1	AZURE DEVOPS ENVIRONMENT SETUP
-----------	--------------------------------

Aim:

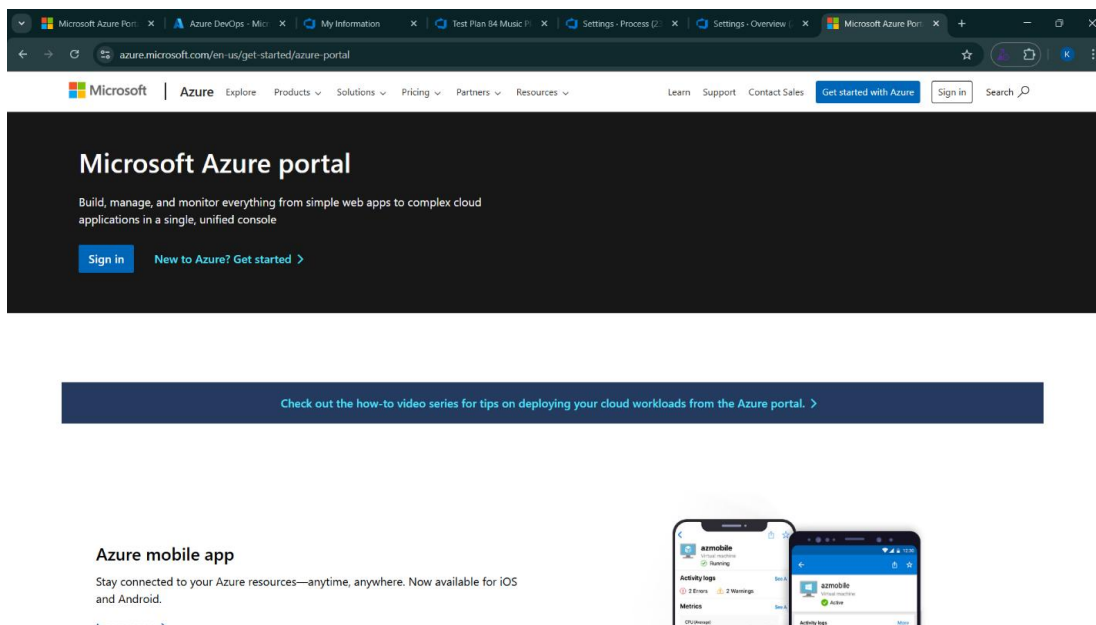
To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

INSTALLATION

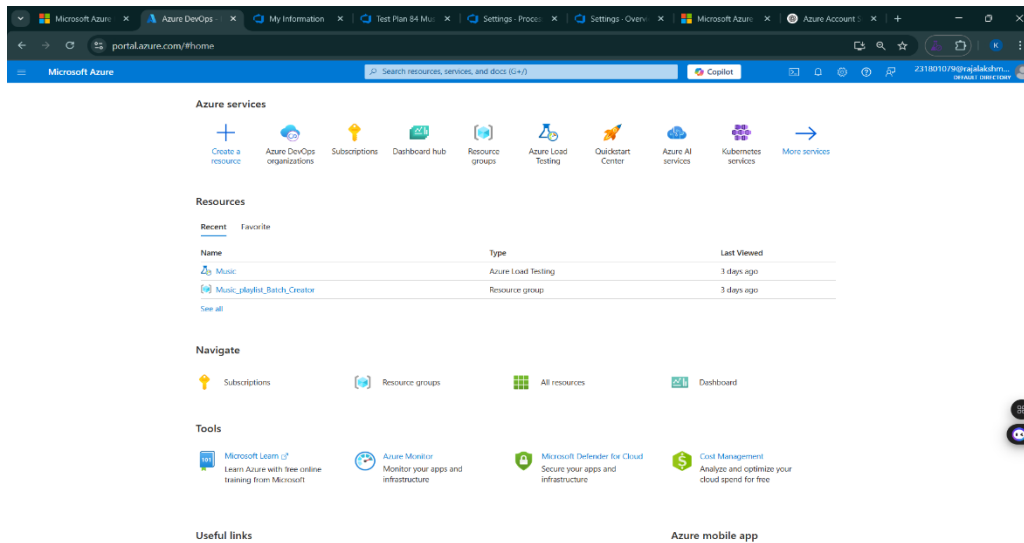
1. Open your web browser and go to the Azure website: <https://azure.microsoft.com/en-us/get-started/azure-portal>.

Sign in using your Microsoft account credentials.

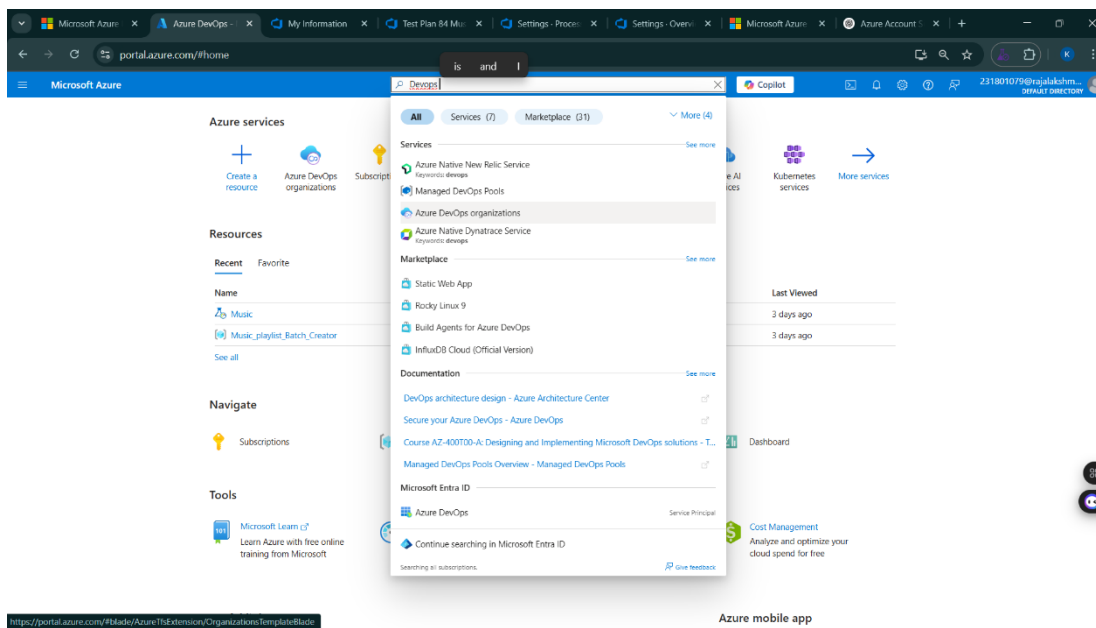
If you don't have a Microsoft account, you can create one here: <https://signup.live.com/?lic=1>



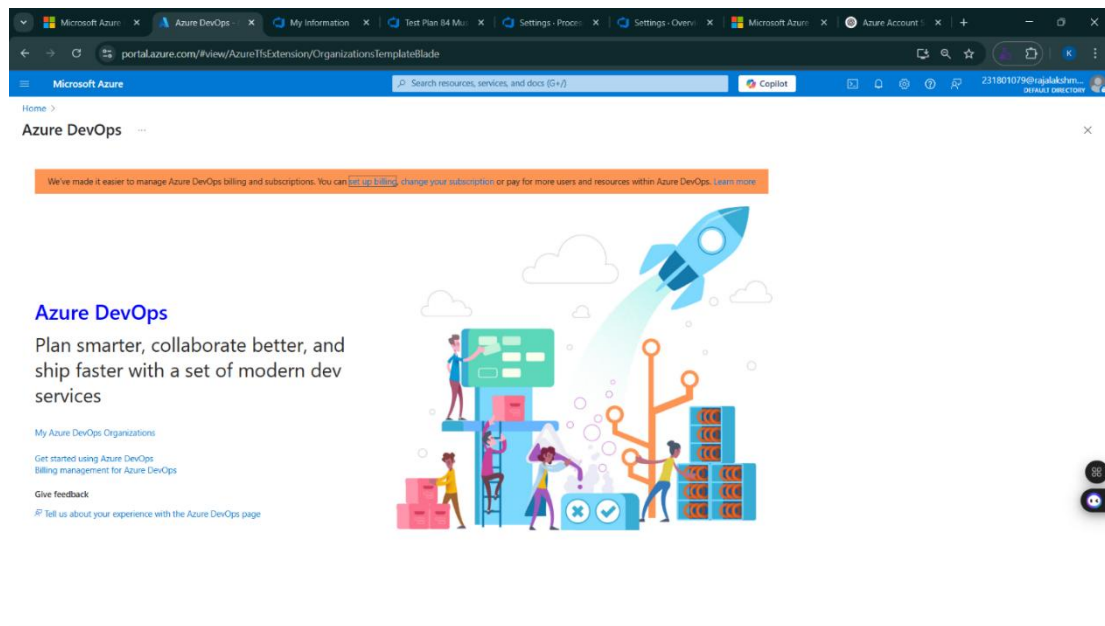
2. Azure home page



3. Open DevOps environment in the Azure platform by typing *Azure DevOps Organizations* in the search bar.



4. Click on the *My Azure DevOps Organization* link and create an organization and you should be taken to the Azure DevOps Organization Home page.



Result:

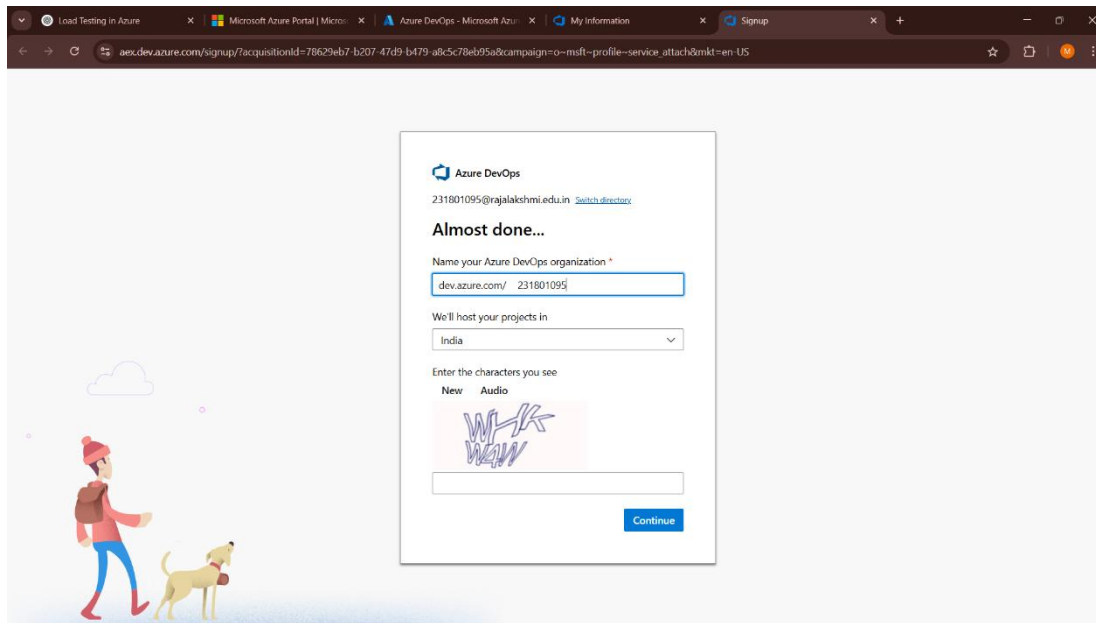
Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

EXP NO: 2	AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT
-----------	---

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

1.Create An Azure Account



2.Create the First Project in Your Organization

- After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.
- On the organization's **Home page**, click on the **New Project** button.
- Enter the project name, description, and visibility options:
 - Name:** Choose a name for the project (e.g., **LMS**).
 - Description:** Optionally, add a description to provide more context about the project.
 - Visibility:** Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).
- Once you've filled out the details, click **Create** to set up your first project.

Create new project



Project name *

Service Management System

Description

Visibility



Private

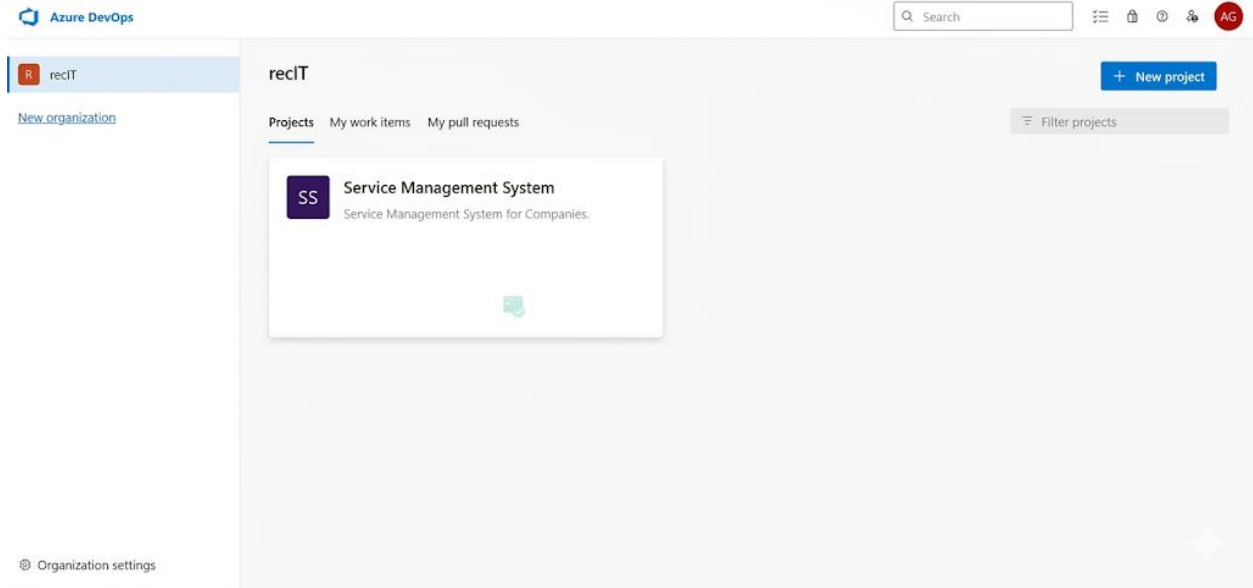
Only people you give access to will be able to view this. Want to create a public project? [Try GitHub](#)

▼ **Advanced**

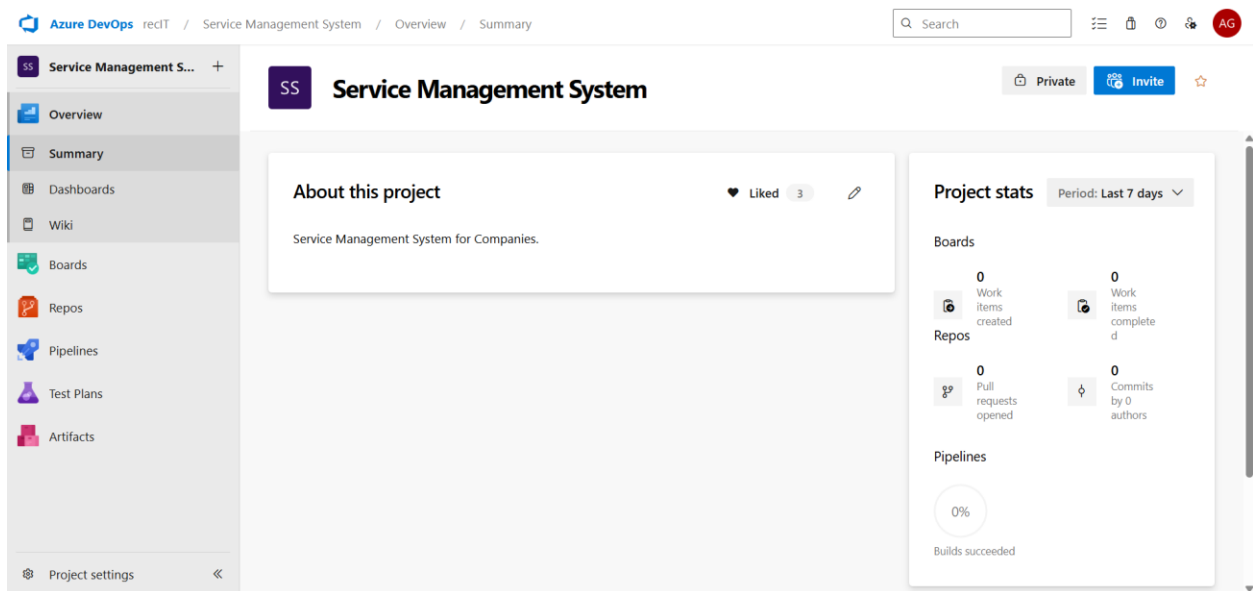
Cancel

Create

3. Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



4. Project dashboard



5.To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.

The screenshot displays the Azure DevOps interface for the 'Service Management System Team'. The 'Boards' section is active, showing a 'Backlog' view. The left-hand navigation menu includes options like Overview, Boards, Work Items, Backlogs, Sprints, Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, and Project settings. The main area shows a list of work items under the 'Backlog' tab. The work items are organized into a table with columns for Work Item Type, Title, State, Story Points, and Value Area. The items include an Epic, several Features, and multiple User Stories, all currently in a 'New' state.

Work Item Type	Title	State	Story Points	Value Area
Epic	Epic 1: User Authentication & Account Management	New		Business
Feature	Feature 1.1: User Registration	New		Business
User Story	User Story 1.1.1: Client Registration	New		Business
User Story	User Story 1.1.2: Registration Performance	New		Business
User Story	User Story 1.1.3: Password Security	New		Business
Feature	Feature 1.2: User Login	New		Business
User Story	User Story 1.2.1: Client Login	New		Business
User Story	User Story 1.2.2: Forgot Password	New		Business
User Story	User Story 1.2.3: Login Security	New		Business
Feature	Feature 1.3: Role Management	New		Business
User Story	User Story 1.3.1: Assign Roles	New		Business
User Story	User Story 1.3.2: Real-Time Role Update	New		Business

Result:

Successfully created an Azure DevOps project with user story management and agile workflow setup.

EXP NO: 3

SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

Aim:

To learn about how to create epics, user story, features, backlogs for your assigned project.

Create Epic, Features, User Stories, Task

The screenshot shows the Azure DevOps interface for the 'Service Management System Team'. The left sidebar contains navigation options: Overview, Boards, Work Items, Backlogs (selected), Sprints, Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, and Project settings. The main area displays the 'Backlog' view with a table of work items. The table has columns for Order, Work Item Type, Title, State, Effort, Business Area, Value Area, and Tags. The work items are organized into a hierarchy: Epic 1 (User Authentication & Account Management) contains Feature 1.1 (User Registration), which contains three User Stories (Client Registration, Registration Performance, Password Security). Epic 1.2 (User Login) and Epic 1.3 (Role Management) are also listed. Below these are Epic 2 (Service Request Management), Epic 3 (Notifications & Alert), and Epic 4 (Reporting & Analytics). The states are mostly 'New', with Epic 3 being 'Active'.

Order	Work Item Type	Title	State	Effort	Busin...	Value Area	Tags
	Epic	👑 Epic 1: User Authentication & Account Management	New			Business	
	Feature	👑 Feature 1.1: User Registration	New			Business	
	User Story	👤 User Story 1.1.1: Client Registration	New			Business	
	User Story	👤 User Story 1.1.2: Registration Performance	New			Business	
	User Story	👤 User Story 1.1.3: Password Security	New			Business	
	Feature	👑 Feature 1.2: User Login	New			Business	
	Feature	👑 Feature 1.3: Role Management	New			Business	
	Epic	👑 Epic 2: Service Request Management	New			Business	
	Epic	👑 Epic 3: Notifications & Alert	Active			Business	
	Epic	👑 Epic 4: Reporting & Analytics	New			Business	

1.Fill in Epics

The screenshot shows the detailed view of 'Epic 1: User Authentication & Account Management' in Azure DevOps. The top bar includes the title, author (Naresh M), and a comment count (1). The main content area is divided into three sections: Description, Planning, and Deployment. The Description section has a text area for adding comments and a link to the Markdown editor. The Planning section includes fields for Priority (2), Risk, Effort, Business Value, Time Criticality, Start Date, and Target Date. The Deployment section contains a link to the Releases page and a link to the Development page. The bottom of the screen shows the project settings and a navigation bar.

Description

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

Planning

Priority: 2

Risk

Effort

Business Value

Time Criticality

Start Date

Select a date...

Target Date

Deployment

To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)

Development

Add link

Link a GitHub [commit](#), [pull request](#) or [branch](#) to see the status of your development. You

2.Fill in Features

The screenshot shows the Azure DevOps interface for a Feature. The breadcrumb navigation is "Azure DevOps" / "recIT" / "Service Management System" / "Boards" / "Backlogs". The feature is titled "22 Feature 1.1: User Registration" and is owned by "Naresh M". It has "0 Comments" and an "Add Tag" button. The feature is in the "New" state and is associated with the "Service Management System" area and iteration. The description is "Enables clients to create an account in the system using email and password, with email confirmation for verification." The planning section includes fields for Priority (2), Risk, Effort, Business Value, Time Criticality, Start Date, and Target Date. The deployment section includes a link to "Releases" and a link to "Deployment status reporting". The development section includes a link to "Add link" and a link to "Link a GitHub commit, pull request or branch to see the status of your development. You".

3.Fill in User Story Details

The screenshot shows the Azure DevOps interface for a User Story. The breadcrumb navigation is "Azure DevOps" / "recIT" / "Service Management System" / "Boards" / "Backlogs". The user story is titled "23 User Story 1.1.1: Client Registration" and is owned by "Naresh M". It has "0 Comments" and an "Add Tag" button. The user story is in the "New" state and is associated with the "Service Management System" area and iteration "Service Management System(Sprint 1)". The description is "As a new client, I want to register an account using my email and password so that I can access the system." The acceptance criteria are: "User can enter email and password to create account.", "Duplicate emails are not allowed.", and "User receives confirmation email upon successful registration." The type is "Functional Requirement (FR)". The planning section includes fields for Story Points, Priority (2), and Risk. The classification section includes fields for Value area and Business. The deployment section includes a link to "Releases" and a link to "Deployment status reporting". The development section includes a link to "Add link" and a link to "Link a GitHub commit, pull request or branch to see the status of your development. You".

Result:

Thus, the creation of epics, features, user story and task has been created successfully.

EXP NO: 4	SPRINT PLANNING
-----------	-----------------

Aim:

To assign user story to specific sprint for the Service Management System Project.

Sprint Planning Sprint 1

Service Management System Team | April 12 - April 27 (11 work days)

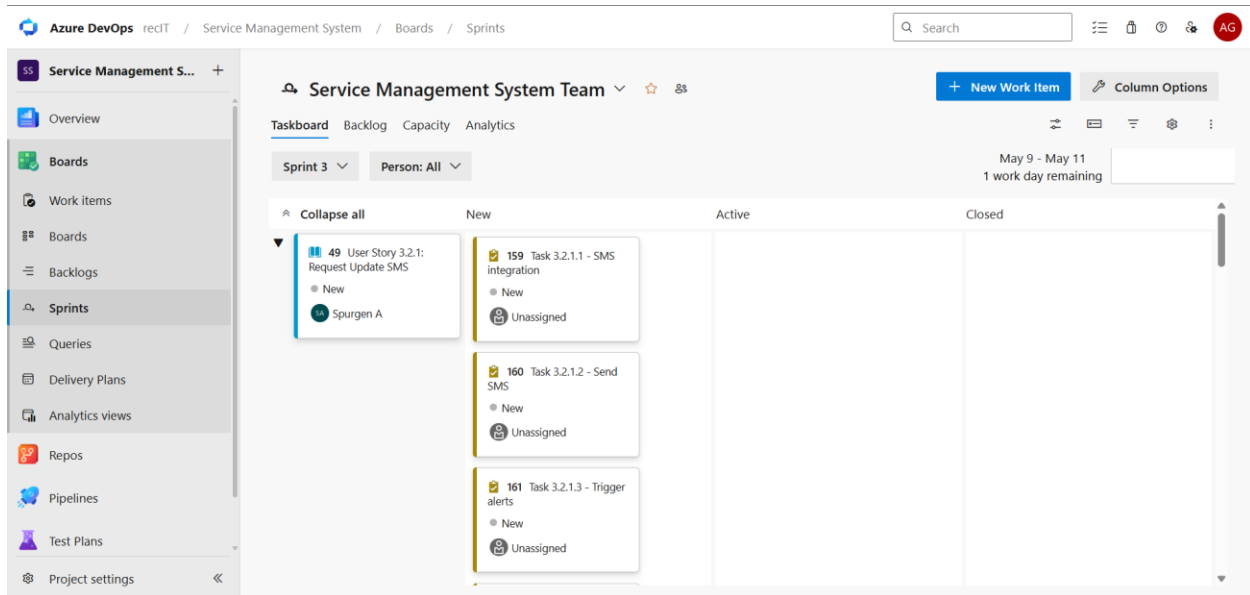
Item	Category	Assignee
23 User Story 1.1.1: Client Registration	User Story	Naresh M
59 Task 1.1.1.2 - Create user database table	Task	Unassigned
60 Task 1.1.1.3 - Develop registration API	Task	Unassigned
61 Task 1.1.1.4 - Implement input validation	Task	Unassigned

Sprint 2

Service Management System Team | May 1 - May 8 (6 work days)

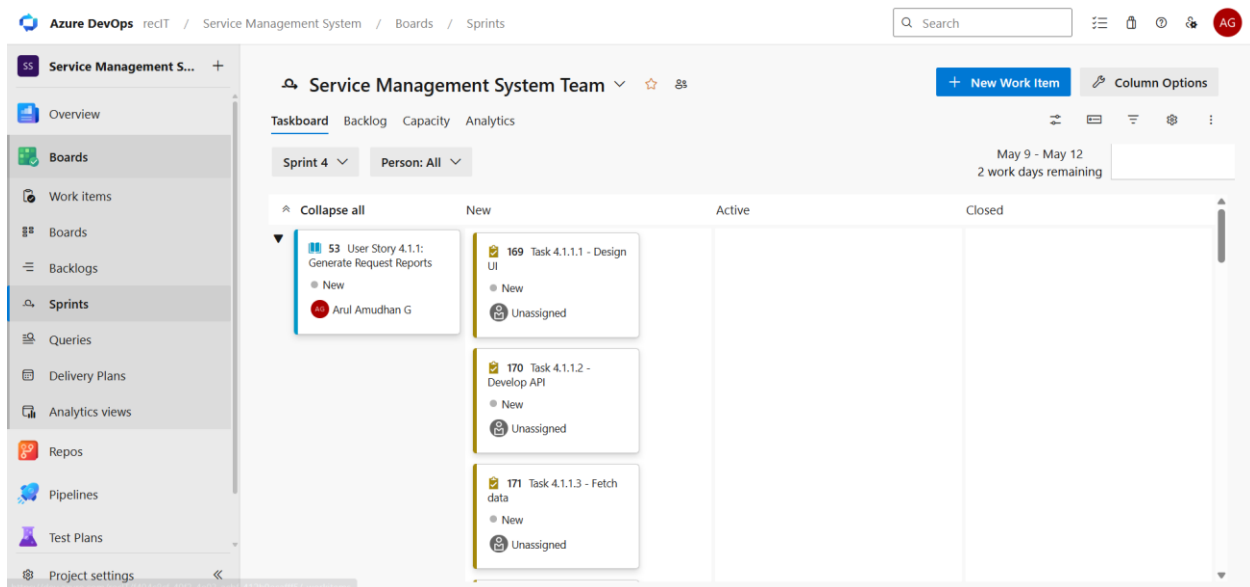
Item	Category	Assignee
35 User Story 2.1.1: Submit Service Request	User Story	Sathish R
107 Task 2.1.1.1 - Design form	Task	Unassigned
108 Task 2.1.1.2 - Create database	Task	Unassigned
109 Task 2.1.1.3 - Develop API	Task	Unassigned

Sprint 3



The screenshot shows the Azure DevOps interface for the 'Service Management System Team'. The left sidebar contains navigation links: Overview, Boards, Work items, Backlogs, Sprints (selected), Queries, Delivery Plans, Analytics views, Repos, Pipelines, Test Plans, and Project settings. The main area displays the 'Sprint 3' board for the period 'May 9 - May 11' with '1 work day remaining'. The board is organized into columns: 'Collapse all', 'New', 'Active', and 'Closed'. Under the 'New' column, there are three items: '49 User Story 3.2.1: Request Update SMS' (assigned to Spurgan A), '159 Task 3.2.1.1 - SMS Integration' (Unassigned), '160 Task 3.2.1.2 - Send SMS' (Unassigned), and '161 Task 3.2.1.3 - Trigger alerts' (Unassigned). The top right includes a search bar, a '+ New Work Item' button, and 'Column Options'.

Sprint 4



The screenshot shows the Azure DevOps interface for the 'Service Management System Team'. The left sidebar is identical to the previous screenshot, with 'Sprints' selected. The main area displays the 'Sprint 4' board for the period 'May 9 - May 12' with '2 work days remaining'. The board is organized into columns: 'Collapse all', 'New', 'Active', and 'Closed'. Under the 'New' column, there are three items: '53 User Story 4.1.1: Generate Request Reports' (assigned to Arul Amudhan G), '169 Task 4.1.1.1 - Design UI' (Unassigned), '170 Task 4.1.1.2 - Develop API' (Unassigned), and '171 Task 4.1.1.3 - Fetch data' (Unassigned). The top right includes a search bar, a '+ New Work Item' button, and 'Column Options'.

Result:

The Sprints are created for the Service Management System Project.

EXP NO: 5	POKER ESTIMATION
------------------	-------------------------

Aim:

Create Poker Estimation for the user stories - Service Management System Project.

Planning Poker Estimation:

User Story 1: Submit Service Request

User Story ID: 01

Title: Submit Service Request

Description: As a Client, I want to submit a service request with details like service type, description, and priority so that the administration can review and assign my request.

Story Point Estimation: 5 Points

Team Poker Votes:

Team Member	Vote (Story Point)
Team Member 1	3
Team Member 2	5
Team Member 3	5
Team Member 4	8

Final Consensus: 5 Points

Reason for Estimation:

- Requires building a dynamic frontend form with validation.
- Backend logic involves mapping the request to the currently authenticated user (req.user.id).
- Database integration with the ServiceRequest model using Mongoose.
- Includes handling different priority levels and service types.

Risks:

- Data validation failures (e.g., empty descriptions).
- Inconsistent priority handling between UI and Backend.

Acceptance Criteria Covered:

- Client can enter serviceType, description, and priority.

- System automatically assigns "Pending Approval" status to new requests.
- Request is correctly linked to the logged-in client's ID.
- Successful submission redirects the user to their dashboard.

The screenshot shows an Azure DevOps work item page for 'USER STORY 47'. The browser tabs at the top include 'Employee Data Preprocess', 'Inbox - 231801079@rajala', 'Microsoft Azure Portal | M', 'Azure DevOps - Microsoft', 'Music Playlist Batch Creat', and 'KARTHIK-231801079/M...'. The address bar shows the URL: `dev.azure.com/231801095/Music%20Playlist%20Batch%20Creator/_backlog/backlog/Music%20Playlist%20Batch%20Creator%20Team/Epics?workitem=47`.

The work item details are as follows:

- USER STORY 47**: As a user i should be able to create audio playlist as i need
- Owner**: Karthick S
- Comments**: 0 Comments, [Add Tag](#)
- Buttons**: Save and Close, Follow, Refresh, Copy, More options
- Status**: Resolved
- Area**: Music Playlist Batch Creator
- Reason**: Code complete and unit t
- Iteration**: Music Playlist Batch Creator/Model
- Updated by**: Mallu karthick Balaji Reddy, Feb 18
- Details**: View icon, 1 icon, 0 icon

The main content area is divided into several sections:

- Description**: Click to add Description.
- Acceptance Criteria**: Click to add Acceptance Criteria.
- Discussion**: Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request. [switch to Markdown editor](#)
- Planning**:
 - Story Points: 3
 - Priority: 2
 - Risk:
- Classification**:
 - Value area: Business
- Deployment**: To track releases associated with this work item, go to [Releases](#) and turn on deployment status reporting for Boards in your pipeline's Options menu. [Learn more about deployment status reporting](#)
- Development**:
 - Add link**: Link an Azure Repos [commit](#), [pull request](#) or [branch](#) to see the status of your development. You can also [create a branch](#) to get started.
- Related Work**:
 - Add link** (dropdown arrow)
 - Parent**: 26 Auto-Playlist Creation Based on user preference, Updated Feb 18, New

Result:

The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 6

DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE

Aim:

To Design a Class Diagram and Sequence Diagram for the given Project.

6A. Class Diagram

Classes:

- User
- ServiceRequest
- Feedback
- Ticket
- ChatMessage

Key Attributes:

User: name, email, password, role (Client, Admin, Developer, Tester), isVerified

ServiceRequest: serviceType, description, priority, status, devStatus, testerStatus, bugsReported, slaDeadline

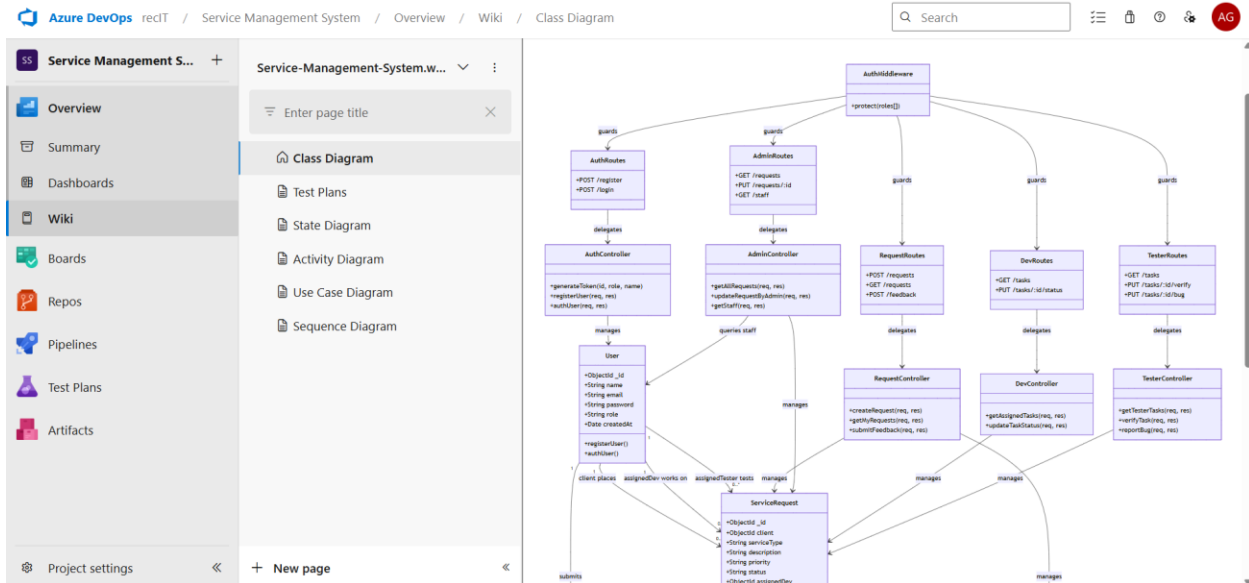
Feedback: rating, comments, createdAt

Ticket: subject, description, status, priority, createdAt

ChatMessage: message, read, createdAt

Relationships:

- User (Client) → creates → ServiceRequest
- User (Admin) → assigns → ServiceRequest
- User (Developer) → works on → ServiceRequest
- User (Tester) → verifies → ServiceRequest
- User (Client) → submits → Feedback
- Feedback → belongs to → ServiceRequest
- User → opens → Ticket
- User → sends → ChatMessage
- ChatMessage → linked to → ServiceRequest



6B. Sequence Diagram

Actors:

Client / Admin / Developer / Tester → Frontend → Server → Database (DB)

Steps:

1. Client enters registration details
2. Frontend sends register request to Server
3. Server saves user in DB
4. Server returns JWT token to Client
5. Client enters login credentials
6. Frontend sends login request to Server
7. Server verifies credentials in DB
8. Server returns JWT token to Client
9. Client submits a new service request
10. Frontend sends service request with token to Server
11. Server saves service request in DB
12. Server returns success response to Client
13. Admin requests dashboard data
14. Server fetches all requests and staff list from DB
15. Server returns requests and staff list to Admin
16. Admin assigns a Developer and Tester to the request
17. Server updates the request assignments in DB
18. Developer requests assigned tasks
19. Server fetches assigned tasks from DB and returns to Developer
20. Developer marks task as "Completed"
21. Server updates devStatus and changes overall status to "In Testing" in DB
22. Tester requests ready-for-testing tasks
23. Server fetches tasks from DB and returns to Tester

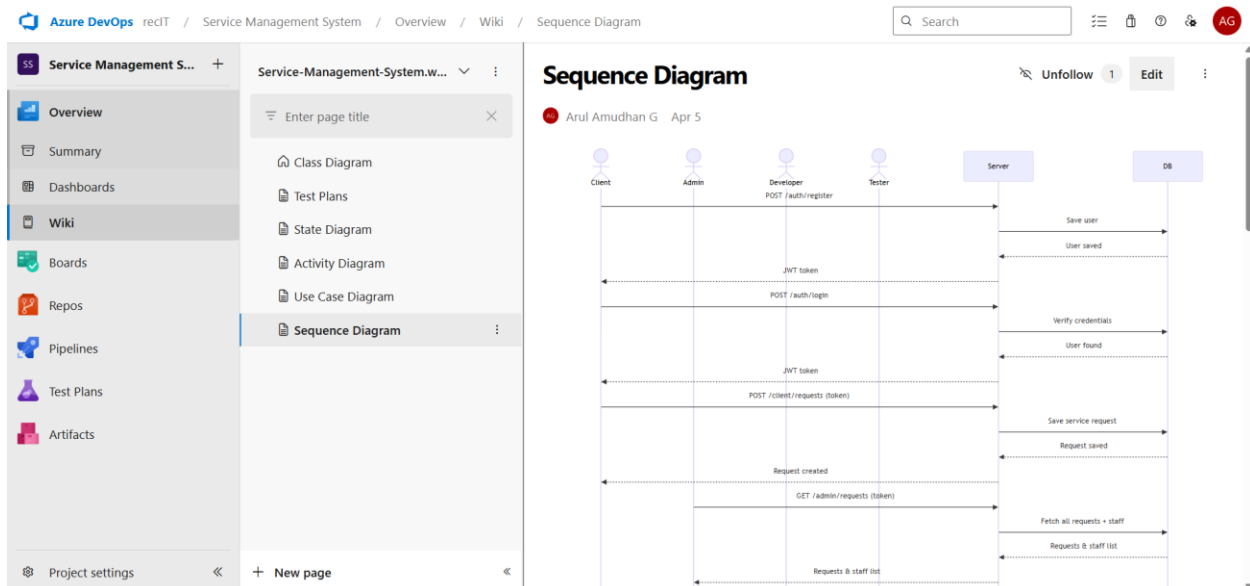
24. Tester evaluates the completed task
25. Admin marks the final request status as "Completed"
26. Server updates final status in DB
27. Client submits feedback for the completed request
28. Server saves feedback in DB
29. Service request lifecycle successful

Condition: Bugs Found

- Tester sends bug report to Server
- Server updates status to "In Development" in DB
- Task is sent back to Developer for fixes

Condition: Verified Successfully

- Tester sends verify request to Server
- Server updates status to "Ready for Delivery" in DB



Result:

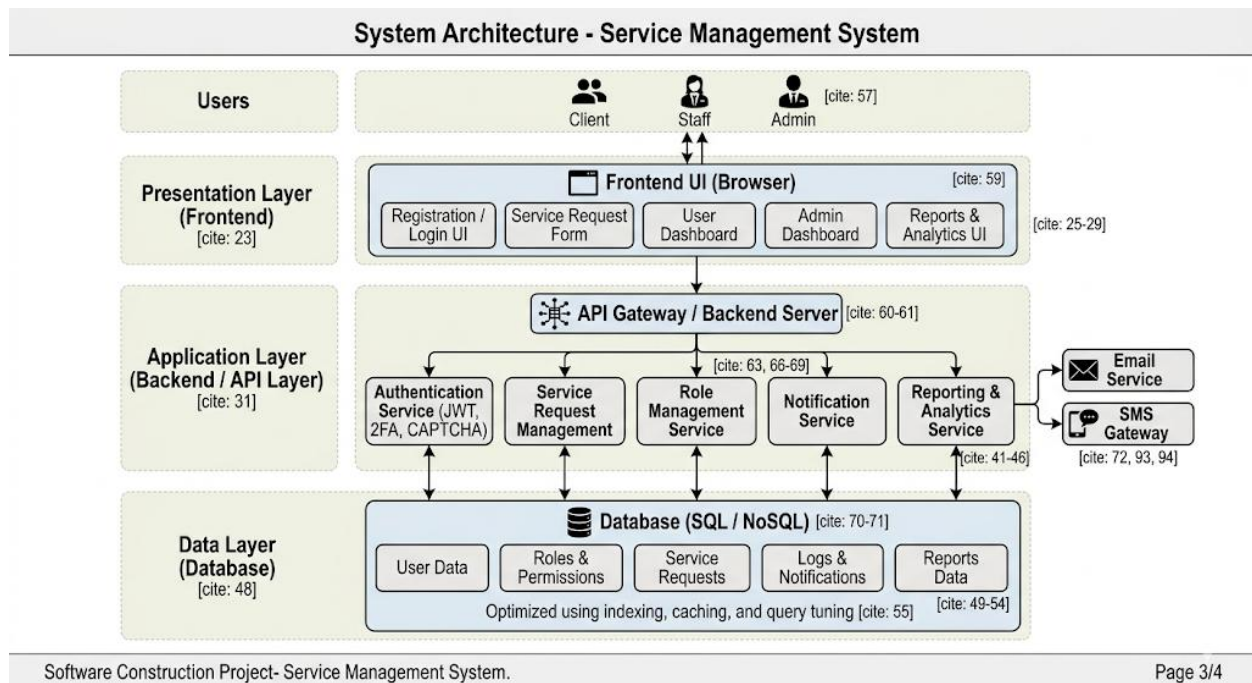
The Class Diagram and Sequence Diagram is designed Successfully for Service Management System Project.

EXP NO: 7	DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE
-----------	---

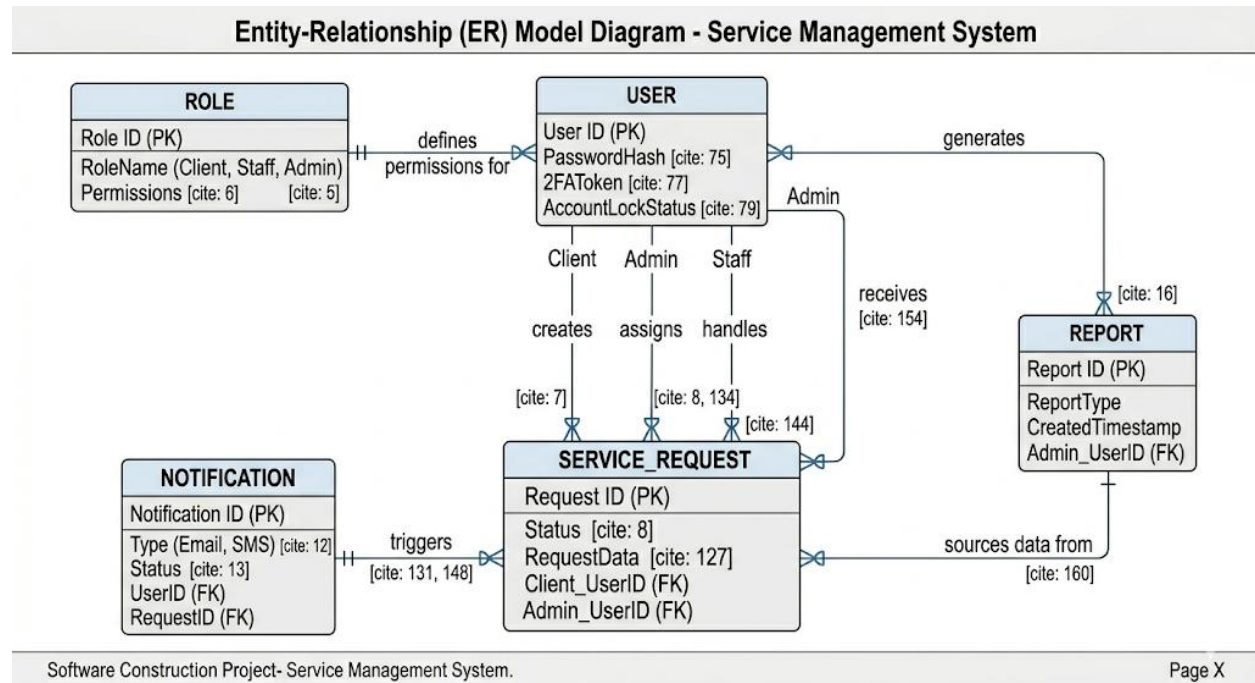
Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

7A. Architectural Diagram:



7B.ER Diagram:



Result:

The Architecture Diagram and ER Diagram is designed Successfully for the Service Management System.

EXP NO: 8	TESTING – TEST PLANS AND TEST CASES
------------------	--

Aim:

Test Plans and Test Case and write two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Case Design Procedure

Procedure

1. Identify system modules such as Login, Registration, Request Management, and Admin Control.
2. Define input values and expected outputs for each module.
3. Create positive and negative test cases.
4. Execute the test cases manually or using testing tools.
5. Compare actual output with expected results.
6. Record defects and update the system if errors occur.

1. Use Clear Naming and IDs

- Test cases are named clearly
- Helps in quick identification and linking to user stories or features.

2. Separate Test Suites

- Grouped test cases based on functionality
- Improves organization and test execution flow in Azure DevOps.

3. Prioritize and Review

- Critical user actions are marked high-priority.
- Reviewed for completeness and traceability against feature requirements.

1.New test plan

The screenshot shows the 'New Test Plan' form in the Azure DevOps interface. The left sidebar contains navigation links: Overview, Boards, Repos, Pipelines, Test Plans (selected), Test plans, Progress report, Parameters, Configurations, Runs, Exploratory sessions, and Artifacts. The main area is titled 'New Test Plan' and contains three input fields: 'Name' with the value 'login page', 'Area Path' with the value 'Service Management System', and 'Iteration' with the value 'Service Management System'. At the bottom right, there are 'Create' and 'Cancel' buttons.

Azure DevOps redT / Service Management System / Test Plans

Service Management S... +

Overview

Boards

Repos

Pipelines

Test Plans

Test plans

Progress report

Parameters

Configurations

Runs

Exploratory sessions

Artifacts

New Test Plan

Name *
login page

Area Path *
Service Management System

Iteration *
Service Management System

Create Cancel

2.Test suite

The screenshot shows the 'Test Suites' view in the Azure DevOps interface. The left sidebar is the same as in the first screenshot. The main area is titled 'Test Suites' and shows a list of suites with 'login' selected. The right sidebar shows the details for the 'login' suite, including a 'Define' tab and a 'New Test Case' button.

Service Management S... +

Overview

Boards

Repos

Pipelines

Test Plans

Test plans

Progress report

Parameters

Configurations

Runs

Exploratory sessions

Artifacts

login

May 3 - May 10 Current

Test Suites

Filter suites by name

login

login (ID: 193)

Define Execute Chart

Add a test case

Use this tab to collate, add and manage test cases

New Test Case

3. Test case

Give two test cases for at least five user stories showcasing the happy path and error scenarios in azure DevOps platform.

Service Management System – Test Plans

USER STORIES

1. As a user, I want to sign up and log in securely so that I can access the Service Management System. (ID: 79)
2. As a user, I need to view all my service requests in one place. (ID: 76)
3. As a user, I should be able to create a service request when needed. (ID: 73)
4. As a user, I should be able to edit, update, and change service request details. (ID: 68)
5. As a user, I need to receive real-time notifications and status updates. (ID: 65)

Test Suites

Test Suit: TS01 - User Login (ID: 86)

1. TC01 – Successful Sign Up

- **Action:**
 - Go to the Sign-Up page.
 - Enter valid name, email, and password.
 - Click "Sign Up".
- **Expected Results:**
 - Sign-Up form is displayed.
 - Fields accept values without error.
 - Account is created, and the user is redirected to the dashboard.
- **Type:** Happy Path

2. TC02 – Secure Login

- **Action:**
 - Go to the Login page.
 - Enter valid email and password.
 - Click on "Login".
- **Expected Results:**
 - Login form is displayed.
 - Fields accept data without error.
 - User is logged in and redirected to the dashboard.
- **Type:** Happy Path

3. TC03 – Sign Up with Existing Email

- **Action:**
 - Go to the Sign-Up page.
 - Enter a name and an already registered email.
 - Click on "Sign Up".
- **Expected Results:**

- Fields accept data.
- Error message "Email already registered" is displayed.
- **Type:** Error Path

4. TC04 – Login with Wrong Password

- **Action:**
 - Go to the Login page.
 - Enter valid email and incorrect password.
 - Click on "Login".
- **Expected Results:**
 - Input is accepted.
 - Error message "Invalid username or password" is shown.
- **Type:** Error Path

Test Suite: TS02 - View Service Requests (ID: 87)

Action:

- Login successfully.
- Navigate to "My Requests".

Expected Results:

- All submitted service requests are displayed clearly.

Type: Happy Path

TC04 – Failed Request Loading

Action:

- Disconnect internet connection.
- Open "My Requests".

Expected Results:

- Requests fail to load.
- Error message "Unable to load requests" is displayed.

Type: Error Path

Test Suit 1:

The screenshot shows the Azure DevOps Test Plans interface. On the left, a sidebar lists navigation options: Overview, Boards, Repos, Pipelines, Test Plans, Test plans, Progress report, Parameters, Configurations, Runs, Exploratory sessions, and Artifacts. The main area is titled 'login (ID: 193)' and has tabs for 'Define', 'Execute', and 'Chart'. Under the 'Define' tab, there is a section 'Test Cases (2 items)' with a 'New Test Case' button. Below this is a table with the following data:

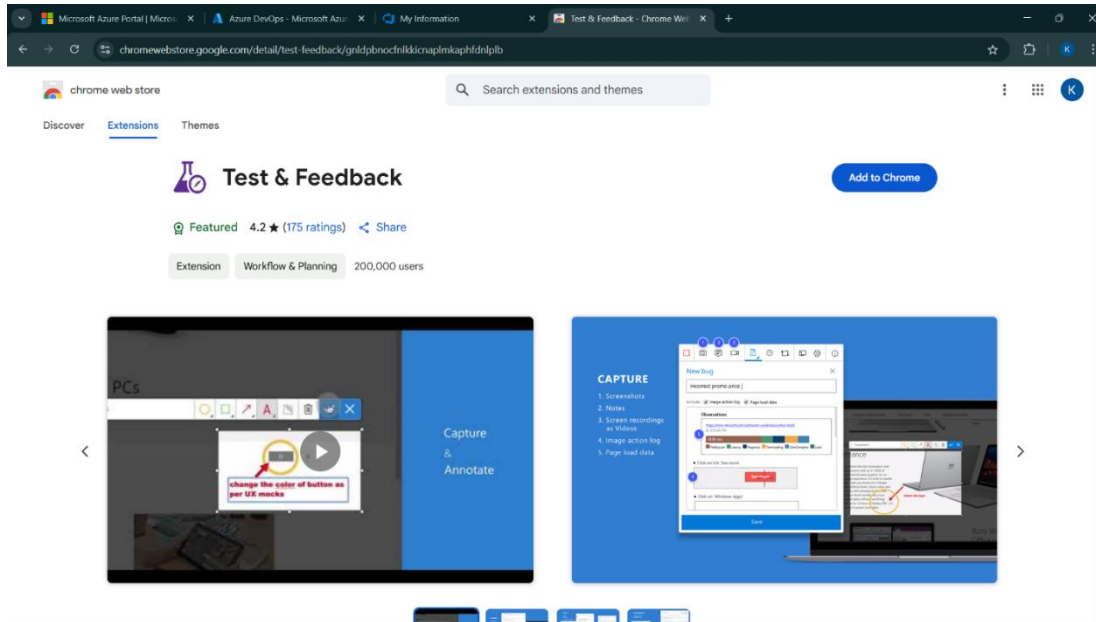
☐ Title	Order	Test Case Id	Assigned To	State
☐ TC01 - Successful User Sign Up	1	197	Naresh M	Design
☐ TC02 - Login with invalid Password	2	198	Naresh M	Design

Test Suit 2:

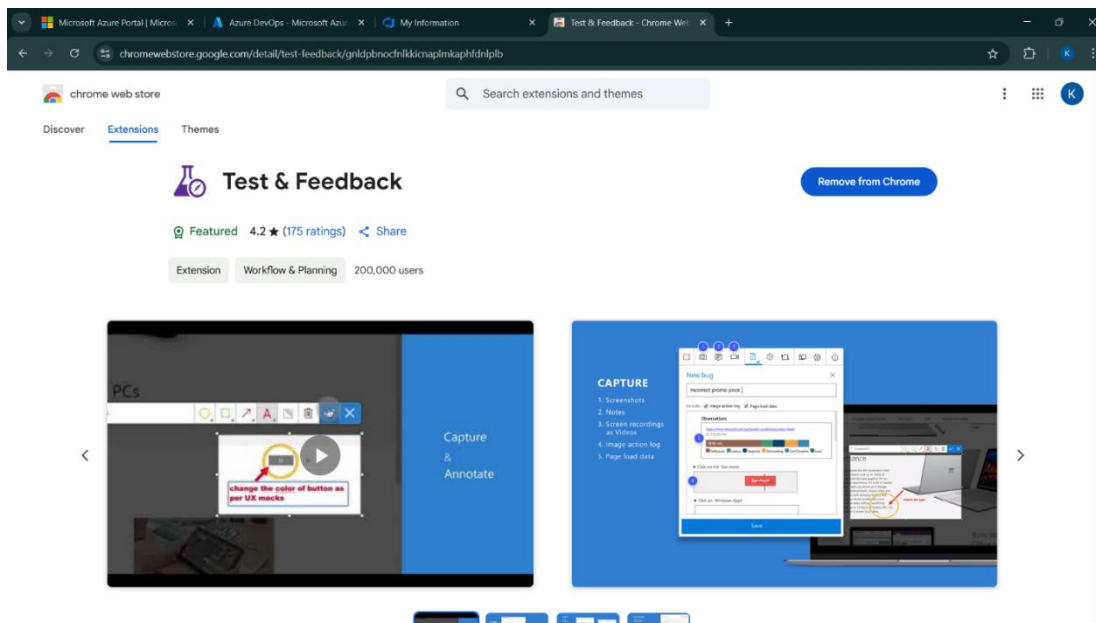
The screenshot shows the Azure DevOps Test Plans interface. On the left, a sidebar lists navigation options: Overview, Boards, Repos, Pipelines, Test Plans, Test plans, Progress report, Parameters, Configurations, Runs, Exploratory sessions, and Artifacts. The main area is titled 'View Service Requests (ID: 201)' and has tabs for 'Define', 'Execute', and 'Chart'. Under the 'Define' tab, there is a section 'Test Cases (2 items)' with a 'New Test Case' button. Below this is a table with the following data:

☐ Title	Order	Test Case Id	Assigned To	State
☐ TC03 - View All Service Requests	1	202	Naresh M	Design
☐ TC04 - Failed Request Loading	2	203	Naresh M	Design

4.Installation of test



5.The test cases are tested successfully and the results are noted.



Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path

EXP NO: 9	LOAD TESTING AND PIPELINES
------------------	-----------------------------------

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Description:

This experiment demonstrates how load testing is performed for the **Service Management System** to evaluate system performance under multiple user requests. The testing checks response time, request handling, and system stability during heavy traffic conditions.

Steps:

Install Load Testing Tool:

1. Download and install Apache JMeter.
2. Open JMeter and create a new Test Plan.
3. Add Thread Group to simulate multiple users.

Create Load Test Configuration

Configure the following components inside JMeter:

- Thread Group
- HTTP Request Sampler
- Listener for Results
- Summary Report

Load Test Configuration

Number of Users (Threads): 50

Ramp-Up Period: 10 Seconds

Loop Count: 5

Server Name: localhost

Port Number: 5000

Request Type: GET

Path: /api/requests

Load Testing Steps

Step 1: Create Thread Group

- Add multiple virtual users.
- Configure ramp-up time and loop count.

Step 2: Add HTTP Request

- Configure API endpoint for testing.
- Select request method such as GET or POST.

Step 3: Add Listener

- Add Summary Report and View Results Tree.
- Monitor request success and failures.

Step 4: Execute Load Test

- Run the test plan.
- Observe response time, throughput, and error rate.

Pipelines:

Description:

This experiment demonstrates how to connect a GitHub-hosted **Service Management System** project with Azure DevOps. The pipeline automatically installs frontend and backend dependencies, builds the application, and publishes artifacts. This helps ensure that every commit triggers automated build validation and smooth deployment.

Steps:

Connect GitHub to Azure DevOps:

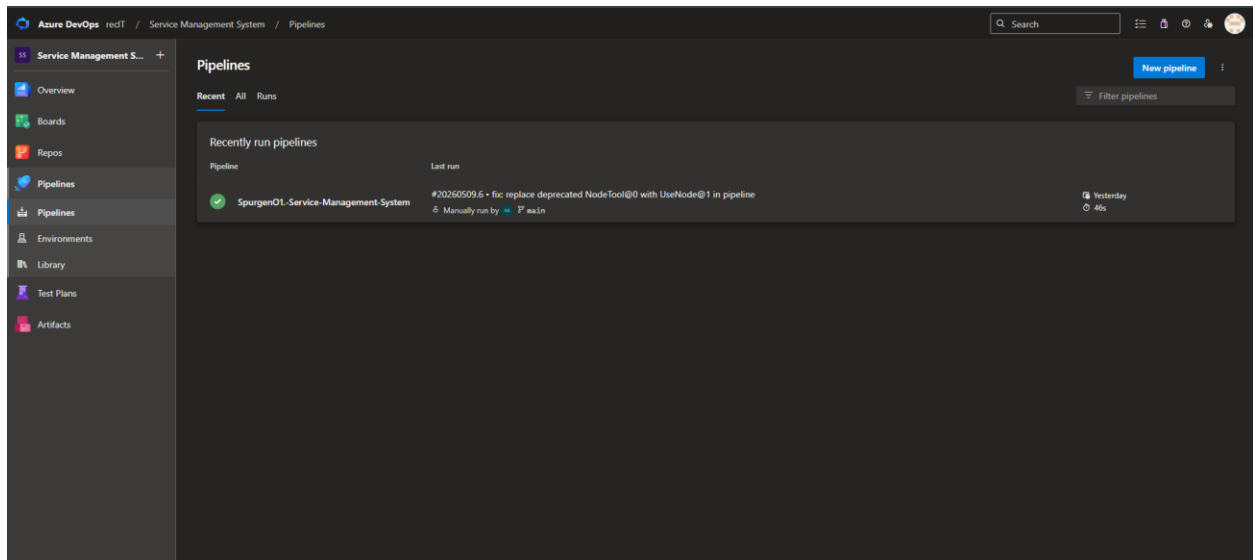
1. Open Azure DevOps and create a new project.

2. Create a new pipeline and select GitHub as the source.
3. Authorize Azure DevOps to access the GitHub repository.
4. Select the Service Management System repository.

Pipeline Tasks Include:

- Connecting GitHub repository with Azure DevOps.
- Installing backend dependencies using npm.
- Building backend services.
- Installing frontend dependencies.
- Building the React/Vite frontend application.
- Running verification steps to ensure successful execution.
- Publishing build artifacts for deployment.

Pipelines:



Azure DevOps recIT / Service Management System / Pipelines / SpurgenO1-Service-Manag... / 20260509.6

Search

This run is being retained as one of 3 recent runs by pipeline. View retention leases

Service Management S... +

- Overview
- Boards
- Repos
- Pipelines
- Pipelines
- Environments
- Library
- Test Plans
- Artifacts

Summary Code Coverage

Manually run by Spurgen A

Repository and version: SpurgenO1-Service-Management-Syst...
Time started and elapsed: Today at 3:53 pm
Related: 0 work items
Tests and coverage: 1 published

View change Parameters

Get started

Stages Jobs

Install & Build
2 jobs completed 43s
1 artifact

Backend (Node.js/Express) 9s
Frontend (React/Vite Dash... 1...

Rerun Stage

Result:

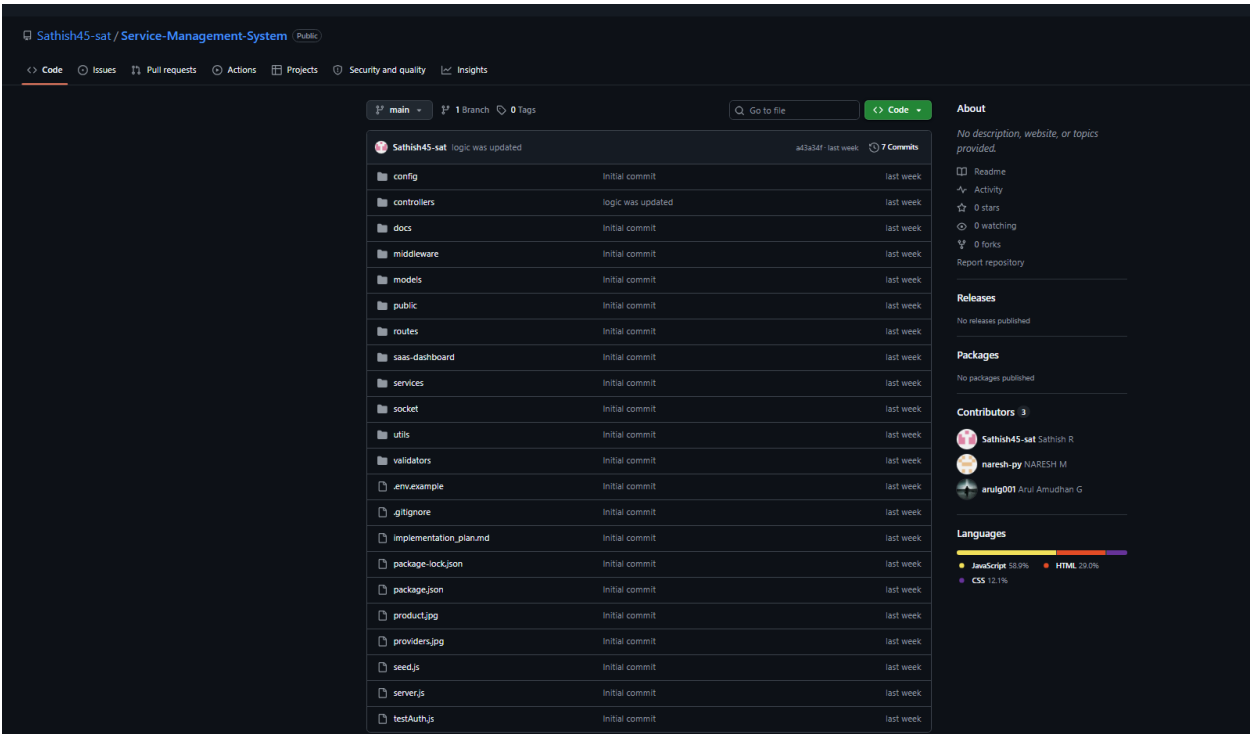
Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

EXP NO: 10	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS
------------	---

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Music Playlist Batch Creator project.

GitHub Project Structure



Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.